

South East Technological University

Department of Computing & Math

Bachelor of Science (Hons) Computer Forensics and
Security

Network and System Security

FYP Report

Fraud Attack and Defence Simulator with AI

Under-Graduate Student

Seán Murphy

Student Number

20101936

Project Sponsors

Sun Life Financial

Wayne Finn, Zohaib Khan

College Supervisor

Jimmy McGibney

April 29, 2026

Table Of Contents

1	Abstract	4
2	Introduction	5
2.1	Motivation and Problem Statement	5
2.2	Research Questions	6
2.3	Project Objectives	6
2.4	Scope and Limitations	7
2.4.1	Domain-Agnostic Framework with Initial Testbed	7
3	Background and Related Work	7
3.1	AI in Cybersecurity	7
3.2	Red Team / Blue Team Methodologies	8
3.3	Adversarial AI and Multi-Agent Systems	8
3.4	Large Language Models and Local Inference	9
3.5	Model Selection	9
3.6	Retrieval-Augmented Generation (RAG)	10
3.7	Vector Databases and ChromaDB	10
3.8	Research Gap	11
4	Methodology	12
4.1	Project Management	12
4.2	Layered Implementation Strategy	12
4.3	Research and Analysis Approach	13
4.4	Technical Stack	13
5	System Design and Architecture	14
5.1	Framework vs. Implementation	14
5.2	Design Philosophy	14
5.3	Overall System Architecture	15
5.4	Core Components	16
5.4.1	Judge	16
5.4.2	Red Team	16
5.4.3	Blue Team	16
5.4.4	Target Environment	16
5.5	Three-Tier RAG Architecture	17
5.5.1	Private RAG	17
5.5.2	Public RAG	18
5.5.3	Shared Battle RAG	18
5.6	Battle Execution Workflow	19
6	Implementation	20
6.1	Infrastructure	20
6.2	Agent Implementation	20
6.2.1	Red Team Pipeline	20
6.2.2	Blue Team Pipeline	21
6.2.3	Judge Pipeline	21
6.3	Agent Self-Assessment	22

6.4	ChromaDB Implementation	22
6.5	Statistics Dashboard	23
6.6	Public Knowledge Base	24
6.7	Technical Challenges Encountered	25
7	Results and Evaluation	27
7.1	Battle Data Overview	27
7.2	Evidence of Agent Learning	28
7.3	RAG Architecture Effectiveness	30
7.4	Framework Validation	32
8	Discussion	33
8.1	Addressing the Research Questions	33
8.2	Key Design Decisions and Trade-offs	34
8.3	Limitations	35
8.4	Relevance to Sun Life Financial	38
9	Conclusion	39
9.1	Summary of Achievements	39
9.2	Future Work	39
9.3	Final Outlook	41
10	References	42
10.1	Additional Resources Consulted	42
10.1.1	Industry Frameworks	42
10.1.2	Key Papers (also cited in text)	43
10.1.3	Video Resources - AI & Machine Learning	43
10.1.4	Video Resources - Fraud Detection & AI Security	43
10.1.5	Video Resources - AI-Powered Threats	43
10.1.6	Technical Documentation	44

1 Abstract

This project presents a Fraud Attack and Defence Simulator with AI, an adversarial multi-agent framework where three competing AI agents continuously battle each other to improve attack and defence capabilities. Sponsored by Sun Life Financial, the project is motivated by a real industry problem: fraud techniques evolve faster than traditional rule-based detection systems can keep up with.

The system runs three specialised agents locally via Ollama on AWS EC2. Red Team (Llama 3.2) generates attacks, Blue Team (Phi4-mini) detects and responds to them, and Judge (Mistral) evaluates each exchange and coordinates the learning cycle. All agent knowledge is stored in a three-tier RAG architecture using ChromaDB: a private knowledge base per agent, a shared battle history accessible to both, and a public knowledge base pre-loaded with 2,500+ entries from MITRE ATT&CK [1], CAPEC [2], CWE [3], OWASP Top 10 [4], and PayloadsAllTheThings [5].

A key contribution of the system is its metacognition module, inspired by the Reflexion framework, which prompts each agent to reflect on battle outcomes and extract strategic insights into their private knowledge base rather than simply logging raw results. Attacks are demonstrated against DVWA (Damn Vulnerable Web Application) to ground simulations in realistic execution.

The framework is intentionally domain-agnostic. Rather than being built for a single threat type, it is designed as a general-purpose simulation engine for any scenario where two sides adapt against each other, an attacker and a defender, a fraudster and a detection system, a red team and a blue team. The web security testbed demonstrates the framework working end-to-end, and extension to insurance fraud detection for Sun Life Financial is the primary planned next application.

Project Repository

- FADS-AI GitHub Repository: https://seanmurphy1479.github.io/fyp_site/

Demo: <https://youtu.be/tlM7QifdLTU>

2 Introduction

2.1 Motivation and Problem Statement

Fraud detection is a constant arms race. Fraudsters learn how detection systems work and adjust their behaviour to stay one step ahead, while financial institutions still rely heavily on static rule-based systems that flag transactions matching known patterns. The moment those patterns become public knowledge, they become obsolete. Manual investigation helps, but it is expensive, slow, and simply cannot scale to the volume of transactions a company like Sun Life Financial processes.

This project came about through an existing relationship with Sun Life Financial. Having completed an internship with the company, the opportunity to take on a sponsored Final Year Project was offered as the placement came to an end. The project brief centred on exploring how AI could be applied to fraud detection and to simulate the interaction between attacker and defender, with the direction and scope shaped through early meetings with industry mentors Wayne Finn and Zohaib Khan.

The problem is getting harder. Fraudsters are increasingly using new tools like AI to generate synthetic identities, fabricate claims, and craft attacks that look convincingly legitimate. Fighting AI-generated fraud with static rules is a losing battle.

This project takes a different approach: let AI agents fight each other. Red Team generates attacks, Blue Team learns to detect them, and both sides improve through competition. Rather than relying on static datasets, the system generates its own training data through live adversarial battles. The key contribution is not any single attack or detection technique, It is the framework itself, a general-purpose simulation engine that can model any scenario where two sides adapt against each other.

Web application security was chosen as the initial testbed because it offers clear, objective metrics an attack either succeeds or not, and has a rich body of public knowledge to draw from. Underpinning the whole design is the concept of adversarial asymmetry: an attacker only needs to find one weakness, while a defender must secure everything and still keep it user friendly. Any realistic simulation has to account for that imbalance.

2.2 Research Questions

This project is structured around four core research questions:

1. **AI Adaptation:** Can AI agents learn from historical battle interactions to measurably improve their attack and defence strategies over time, without any changes to their underlying models?
2. **Knowledge Architecture:** How does a three-tier RAG architecture separating private agent memory, shared battle history, and public domain knowledge support knowledge retention and strategic adaptation in an adversarial multi-agent system?
3. **Framework Generality:** Can a single adversarial framework simulate meaningfully different attack and defence domains by substituting only the target environment and knowledge base, leaving the core agent logic unchanged?
4. **Asymmetric Evolution:** Given that attackers and defenders operate under fundamentally different constraints, how do asymmetric agent objectives influence co-evolution and does competition drive continued improvement or eventual stagnation?

2.3 Project Objectives

- Implement a three-tier RAG architecture providing persistent, separated knowledge for private agent memory, shared battle history, and public domain knowledge
- Build Red Team and Blue Team agents capable of evolving their attack and defence strategies across repeated adversarial battles
- Develop a metacognition module enabling agents to reflect on battle outcomes and develop their own private knowledge bases
- Design a modular, domain-agnostic framework where the target environment and knowledge base can be substituted without changing the core agent logic
- Quantify whether iterative AI-to-AI battles produce measurable improvement in attack sophistication and defensive effectiveness
- Deploy the full system on AWS EC2 with local LLM inference via Ollama and deliver a working prototype with a real-time statistics dashboard

2.4 Scope and Limitations

The scope of this project covers the design and implementation of the core adversarial framework: the domain-agnostic battle orchestration layer, the three-tier RAG architecture with agent self-assessment, the Judge AI responsible for scoring and feedback, and the full AWS deployment. Web application vulnerabilities specifically SQL injection, XSS, and path traversal serve as the initial testbed for validating the framework end-to-end.

There are some honest limitations worth stating upfront. The system runs on AWS Learner Lab infrastructure without GPU instances, meaning all LLM inference runs on CPU via Ollama. This keeps costs within budget constraints but limits battle throughput and model size. Framework extensibility to other domains such as insurance fraud is demonstrated architecturally and argued theoretically, but has not been empirically tested that remains future work. The system is also a research prototype rather than a production-ready tool, and should be evaluated as such.

2.4.1 Domain-Agnostic Framework with Initial Testbed

The primary contribution of this project is the adversarial framework itself, not a fraud detector for any specific domain. The framework is designed so that the core engine the agents, the battle orchestration, the RAG architecture, and the metacognition module remains unchanged regardless of what is being simulated. Swapping the target environment and the public knowledge base is all that is required to move from one domain to another.

Web application security was chosen as the initial testbed for practical reasons. Attack outcomes are objective: a SQL injection either extracts data or it does not. There is a rich body of public knowledge to populate the knowledge base with, from OWASP Top 10 to MITRE ATT&CK. And the feedback loop is fast enough to run meaningful numbers of battles within an academic timeline. It is a proving ground for the framework.

3 Background and Related Work

3.1 AI in Cybersecurity

Traditional cybersecurity relies on signature-based detection and static rule systems if traffic matches a known bad pattern, it gets flagged. This works well enough against known threats, but falls apart against novel attacks. Zero-day vulnerabilities and adversarial evasion techniques exist precisely because attackers know how these systems work and design around them.

Machine learning has improved things, particularly in anomaly detection, but it introduces its own challenges. Models need labelled training data, which is expensive to produce and goes stale quickly as attack patterns evolve. Attackers are no longer just humans exploiting known weaknesses they are increasingly using AI to generate convincing fraudulent activity at scale, making static detection approaches even harder to rely on.

3.2 Red Team / Blue Team Methodologies

Red team and blue team exercises are a well established approach to security testing. A red team attempts to break into a system while a blue team defends it, with the goal of finding weaknesses before real attackers do. In practice these exercises are guided by frameworks like MITRE ATT&CK [1] and OWASP Top 10 [4], which catalogue known attack techniques and vulnerabilities.

The problem with existing automated tools like Metasploit and OWASP ZAP is that they operate on static, predefined attack patterns. They can replay known attacks, but they cannot adapt. This framework takes a different approach: rather than scripting attacks in advance, AI agents use their knowledge to generate and adapt their strategies autonomously based on what has and has not worked in previous attempts.

3.3 Adversarial AI and Multi-Agent Systems

The idea that competition drives improvement in AI systems is not new. Generative Adversarial Networks, introduced by Goodfellow et al. in 2014 [6], showed that two neural networks competing against each other one generating, one discriminating could produce far better results than either could achieve alone. The same principle appears in multi-agent research: OpenAI’s Hide-and-Seek experiments [7] and DeepMind’s AlphaGo [8] both demonstrated that competitive self-play leads to sophisticated emergent strategies that would be difficult to design manually.

This project applies the same core idea, but using LLM-based agents rather than neural network generators. The advantage of LLMs here is that agents reason and communicate in natural language, making their strategies interpretable and their decisions explainable. The framework extends these principles into cybersecurity by adding retrieval-augmented memory, so agents do not just compete they remember, reflect, and adapt based on everything that has happened before.

3.4 Large Language Models and Local Inference

Large Language Models are transformer-based models capable of generating and reasoning about natural language. In this framework they are the core capability behind each agent. Red Team uses one to generate attacks, Blue Team to analyse and respond, and Judge to evaluate the exchange.

There are two broad approaches to running LLMs: cloud-based inference through APIs like OpenAI or Anthropic, and local inference where models run entirely on your own hardware. Cloud inference is convenient but introduces per-token costs, rate limits, and means that all data including attack payloads and agent reasoning leaves your environment. For a project running continuous adversarial battles within its own environment, neither the cost nor the external dependency is suitable.

Local inference via Ollama solves this. Ollama runs open-source models entirely on-device with no API keys and no cost per query. It also abstracts away model management downloading, running, and querying different models all goes through the same consistent local API regardless of model size or architecture. For this project it means all three agents run fully within the AWS environment, keeping inference costs fixed and the system self-contained.

3.5 Model Selection

Three models were selected, one per agent role, based on capability fit and resource constraints. All three run locally via Ollama on AWS EC2 with no GPU.

Llama 3.2 is Meta’s open-source model with strong reasoning and instruction-following, selected for its performance on open-ended generative tasks which suits the Red Team’s job of producing creative and varied attack strategies.

Mistral was selected for its strong general reasoning and consistent instruction adherence, making it well suited to the structured task of scoring both agents and producing reliable battle summaries.

Phi4-mini is Microsoft’s compact model and the result of a mid-project migration. The original selection was Phi-3 Mini, but it was replaced when it became clear it lacked the tool calling support required for Blue Team’s role. Phi4-mini was selected as its replacement. The migration and the technical challenges it introduced are covered in the Implementation section.

3.6 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation [9] combines the generative capability of LLMs with dynamic external retrieval. Rather than relying solely on what a model learned during training, a RAG system queries a vector database at runtime and feeds relevant context directly into the prompt. This means agents can reason about up-to-date, domain-specific knowledge without expensive model retraining, and reduces hallucination in knowledge-intensive tasks important when agents are reasoning about specific attack techniques and defences.

RAG is widely used in chatbots and document question-answering systems, but applying it to adversarial security with a multi-tier architecture is less explored. Self-RAG [10], proposed by Asai et al. in 2023, extends the concept further by having agents reflect on whether retrieval is actually necessary and critique their own outputs rather than blindly retrieving on every query. This idea of self-reflective retrieval directly influenced the metacognition module in this project, where agents assess their own battle performance before deciding what is worth storing in their private knowledge base.

3.7 Vector Databases and ChromaDB

RAG systems require a way to store and retrieve knowledge semantically not by exact keyword match, but by meaning. Vector databases handle this by storing text as high-dimensional numerical embeddings, allowing retrieval of conceptually similar content even when the wording differs.

Several options were considered:

- **FAISS** (Meta) - fast and efficient but in-memory only, no built-in persistence, requires manual index management
- **Pinecone** - managed cloud service with good performance but requires a paid tier and data leaves the local environment
- **Weaviate** - feature-rich but heavyweight, with significant setup overhead for this project
- **ChromaDB** - lightweight, open-source, Python-native, built-in persistence, runs fully locally with no external dependencies

ChromaDB was selected because it fits within the project constraints and supports multiple isolated collections natively, which maps directly to the three-tier RAG design each tier is simply a separate ChromaDB collection, all running within the AWS environment.

3.8 Research Gap

Existing bidirectional adversarial systems like MITRE CALDERA and Microsoft CyberBattleSim automate red and blue team operations, but both rely on reinforcement learning over predefined action spaces. Neither agent reasons about strategy in natural language they select from fixed action sets rather than generating novel approaches. This project uses LLM-based agents that adapt through language reasoning, producing strategies that are both flexible and interpretable.

Reviewed RAG implementations in cybersecurity CyRAG [11], CyKG-RAG [12], TechniqueRAG [13], and CyberRAG [14] each use a single unified knowledge base for defensive threat intelligence. None apply tiered knowledge separation across competing adversarial agents where each side maintains its own private memory invisible to the other.

Self-RAG [10] and Reflexion [15] demonstrate that agents can improve through self-assessment, but both are applied to single cooperative agents working toward a shared goal. This project applies the same self-reflective principle within a competing multi-agent system, where two adversarial agents independently curate asymmetric private knowledge bases what Red Team knows, Blue Team does not, and vice versa.

Finally, the framework runs entirely on commodity cloud hardware using open-source models via Ollama, without GPU clusters or proprietary APIs.

4 Methodology

4.1 Project Management

The project followed an Agile approach, working in two-week development cycles. This made it easier to handle unexpected technical issues such as the mid-project model migration from Phi-3 Mini to Phi4-mini without derailing the overall timeline.

Regular meetings kept the project aligned on two fronts. Weekly meetings with supervisor Jimmy McGibney tracked academic progress and milestone completion. Monthly meetings with Business and Industry mentors at Sun Life Financial ensured the work remained relevant to the industry problem the project is motivated by.

4.2 Layered Implementation Strategy

Development was structured in three progressive layers to manage risk and ensure a working system at every stage.

- **Layer 1 - Core Framework:** Basic agents, battle orchestration, logging, and baseline metrics. This included establishing the agent system prompts, enforcing structured output formats, and getting a stable adversarial loop running end-to-end before any advanced features were added.
- **Layer 2 - Expanded Features:** Three-tier RAG architecture with ChromaDB, SentenceTransformers embeddings, the Flask statistics dashboard, and the public knowledge base populated with 2,529 entries from MITRE ATT&CK, CAPEC, CWE, OWASP Top 10, and PayloadsAllTheThings.
- **Layer 3 - Advanced Features:** Agent metacognition and self-assessment for private RAG curation, attack diversity enforcement with Judge penalty mechanisms, DVWA integration for realistic attack execution, and the mid-project migration from Phi-3 Mini to Phi4-mini to support tool calling.

This structure meant that even if later layers hit problems, the core system would still be functional and demonstrable.

4.3 Research and Analysis Approach

Evaluation draws on both quantitative and qualitative analysis.

On the quantitative side, the system tracks attack success and failure rates, win/loss ratios between agents, score trends across battle rounds, and the growth of RAG collections over time. These metrics aim to provide objective evidence of whether agent performance changes over repeated battles.

On the qualitative side, agent reasoning logs are examined to understand how strategies evolve across battles. Being able to read the agent reasoning directly provides insight into what context is being used, how decisions are being made, and whether the self-assessed insights stored in private RAG differ meaningfully from raw battle logs. This helps determine what is actually happening inside the framework and how the agents are approaching each battle.

4.4 Technical Stack

- **Cloud:** AWS EC2 (t3.large instances), AWS S3 for storage and logging
- **Access & Persistence:** AWS SSM Session Manager for instance access, tmux for persistent process management across sessions
- **AI Models:** Llama 3.2 (Red Team), Phi4-mini (Blue Team), Mistral (Judge) all via Ollama for local inference
- **Knowledge Management:** ChromaDB for vector storage, SentenceTransformers (all-MiniLM-L6-v2) for embedding generation
- **Attack Target:** DVWA (Damn Vulnerable Web Application) for realistic attack execution
- **Web Layer:** Flask for the statistics dashboard and target application
- **Visualisation:** Chart.js for real-time battle monitoring
- **Infrastructure:** AWS CDK for provisioning
- **Development:** Python, Flask, Git

5 System Design and Architecture

5.1 Framework vs. Implementation

This is a framework project, not a single-purpose security tool. The distinction matters because it shapes every design decision made throughout the build.

The system has two conceptual layers. The framework layer stays consistent regardless of what is being simulated this includes the three-tier RAG architecture, the Red Team, Blue Team and Judge agents, the battle orchestration engine, scoring, metrics, and AWS infrastructure.

The domain layer is what gets swapped. The target application, the attack patterns, the public knowledge base content, and the success/failure criteria are all domain-specific. Changing these is enough to move the framework from web security to insurance fraud, or any other scenario where two sides are adapting against each other.

5.2 Design Philosophy

Three principles guided the design of the framework throughout development.

Separation of knowledge. Private, Public, and Shared RAG tiers model the information asymmetry that exists in real adversarial scenarios. An attacker does not have access to a defender's playbook, and vice versa. The architecture reflects that each agent builds its own private knowledge that the other side cannot see.

Observable learning. The Judge provides objective scoring after every battle, creating a measurable record of how both agents perform over time. This makes any improvement or lack of it visible and traceable rather than something that has to be inferred.

Domain-agnostic design. Every component in the framework layer is parameterised rather than hardcoded to a specific attack type or domain. The goal was to build something that could be pointed at a different problem without having to be rebuilt from scratch.

5.3 Overall System Architecture

The system is deployed across multiple AWS EC2 instances, with each agent running on its own instance alongside a separate instance for the target application and dashboard. Agents communicate over internal VPC IP addresses rather than public endpoints, keeping all battle traffic within the private network.

ChromaDB runs on the Judge instance and serves as the central knowledge store for all three RAG tiers. Each agent instance runs Ollama locally for LLM inference, meaning no model traffic leaves the instance it originated from. The Flask dashboard runs alongside the target application and provides a real-time view of battle progress and statistics.

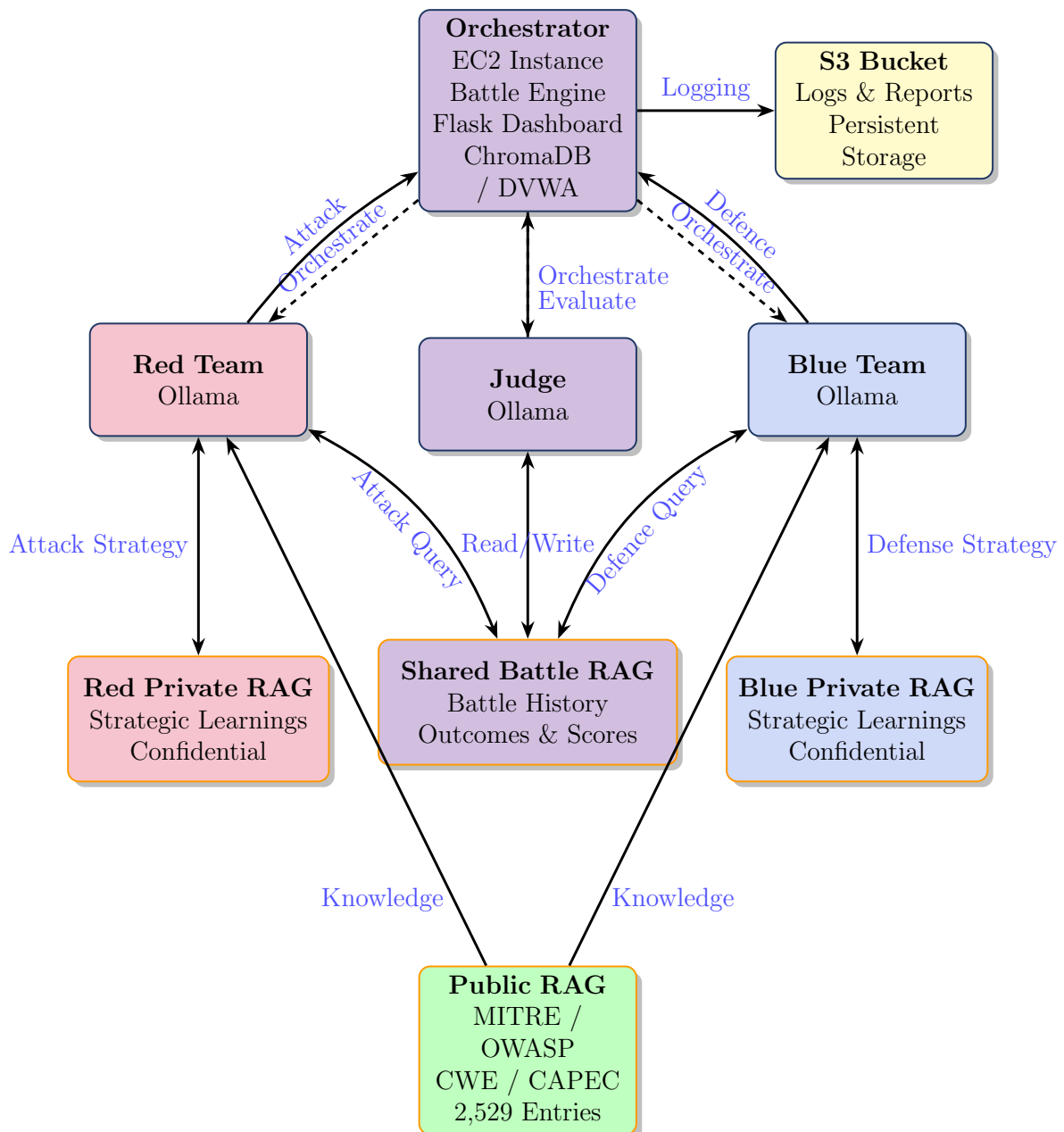


Figure 1: Framework Architecture

5.4 Core Components

The framework is built around three specialised AI agents, each with a distinct role in the adversarial simulation. This section describes what each agent is responsible for and how they interact within the system.

5.4.1 Judge

The Judge runs on its own dedicated EC2 instance and is responsible for coordinating the battle lifecycle timing, sequencing, and round management. After each exchange between Red Team and Blue Team, the Judge evaluates how effective the attack was and how well the defence responded, assigning scores to both agents based on configurable scoring rules.

Once scoring is complete, the Judge writes a summary of the battle outcome to the Shared Battle RAG so both agents can learn from it in subsequent rounds. It also prompts both agents to perform self-assessment, asking each one to reflect on the outcome and decide what strategic insight is worth storing in their private knowledge base.

5.4.2 Red Team

The Red Team runs on its own EC2 instance with Ollama, allowing any supported model to be assigned to the role. Before generating each attack, it queries all three RAG tiers its private strategic learnings, the shared battle history, and the public knowledge base to inform its approach based on what has worked before and what the current target is vulnerable to.

The Red Team has the ability to generate attacks across five categories: SQL injection, XSS, path traversal, authentication bypass, and command injection. Attacks are executed against DVWA via HTTP requests, giving the system real application responses to reason about. Over repeated battles the Red Team adapts its strategy based on what succeeded and what was detected and blocked by Blue Team.

5.4.3 Blue Team

The Blue Team runs on its own EC2 instance with Ollama, with any supported model assignable to the role. After each Red Team attack, the Blue Team receives the HTTP traffic logs generated by the exchange and is tasked with analysing them to classify the attack type and severity and recommend an appropriate mitigation response.

Before generating its response, the Blue Team queries the shared battle history and its own private knowledge base, building up detection patterns over repeated battles based on what it has seen before. The goal is that over time the Blue Team becomes better at recognising attack patterns and responding appropriately, not just reacting to each attack in isolation.

5.4.4 Target Environment

The target environment is DVWA (Damn Vulnerable Web Application), a deliberately vulnerable web application widely used in security education and penetration testing

research. It was selected because it provides a well documented, standardised attack surface with clearly defined vulnerability categories, making it straightforward to validate whether attacks are succeeding and defences are responding correctly.

DVWA runs on the Orchestrator EC2 instance alongside the battle engine and dashboard. It contains openly vulnerable SQL injection points, weak authentication, and unvalidated queries a realistic attack surface for the Red Team to operate against. The security level is configured to low during testing to ensure the full range of attack types can be executed and evaluated without additional hardening interfering with the simulation. Every attack generates Apache logs that the Blue Team analyses to build its response.

The target environment is the only domain-specific component in the framework. Everything else the agents, the RAG architecture, the battle engine stays the same regardless of what is being simulated. Swapping the target environment, the public knowledge base, and the agent prompts is all that is needed to transition the framework to a different domain entirely.

5.5 Three-Tier RAG Architecture

The knowledge architecture is one of the core contributions of this project. Rather than giving all agents access to a single shared knowledge base, the system separates knowledge into three distinct tiers, each serving a different purpose in the learning cycle.

5.5.1 Private RAG

Each agent has its own private knowledge base that the other team cannot access. Rather than being populated automatically with raw battle logs, the private RAG is filled through agent self-assessment after each battle, the agent reflects on the outcome and decides what is genuinely worth remembering.

For the Red Team this means storing things like successful attack patterns, evasion strategies that bypassed Blue Team detection, and techniques that consistently produced results. For the Blue Team it means storing detection signatures that worked, response patterns that successfully identified and classified attacks, and insights about what attack variations to look out for.

The result is that the private RAG functions as each agent's strategic memory rather than a battle archive. The knowledge stored there is curated and meaningful, not just a log of everything that happened. This is a deliberate design decision an agent that retrieves genuinely useful past insights will make better decisions than one retrieving noise.

5.5.2 Public RAG

The public knowledge base comes pre-populated with established industry security knowledge and remains read-only throughout the simulation, agents can retrieve from it but never write to it. It contains 2,500+ entries sourced from MITRE ATT&CK, CAPEC, CWE, OWASP Top 10, and PayloadsAllTheThings, covering everything from high-level attack tactics down to concrete exploitation payloads.

Both agents access the same public knowledge base independently. Red Team uses it to inform attack generation querying for relevant techniques, known payloads, and exploitation patterns. Blue Team uses it to inform detection and response querying for weakness descriptions, remediation guidance, and indicators of compromise. The same knowledge base serves both sides, but each agent retrieves different context from it depending on what it is trying to do.

5.5.3 Shared Battle RAG

The shared knowledge base is accessible by both agents and the Judge, and is the primary driver of the arms race dynamic. After every battle round, the Judge writes a summary of the exchange to the shared RAG what attack was used, whether it succeeded, how the Blue Team responded, and the scores assigned to both sides.

Both agents query this history before each new battle. Red Team uses it to see which attacks have worked in the past and which have been consistently detected, informing what to try next. Blue Team uses it to recognise patterns in Red Team behaviour and anticipate what might be coming. The shared history means that over time both agents are adapting not just to the last battle but to the entire history of the simulation.

Unlike the private RAG, the shared battle RAG gives neither team an exclusive advantage both sides see the same history. The asymmetry comes from what each agent does with that information, which is shaped by their private knowledge and their distinct objectives.

5.6 Battle Execution Workflow

Each battle round follows the same sequence regardless of the domain being simulated. Both agents operate through a three-stage pipeline before the round is evaluated and closed.

Red Team begins with a planning stage, querying the shared battle history to review recent attacks and enforcing diversity rules if the same attack category has been used repeatedly, the agent is required to switch. It then gathers targeted knowledge from all three RAG tiers, retrieving private strategic learnings, relevant public knowledge from MITRE and OWASP, and shared battle history. In the final stage it generates a payload and executes it against DVWA via tool calling using one of six available attack tools: `execute_sql_injection`, `execute_xss_attack`, `execute_command_injection`, `execute_file_inclusion`, `execute_file_upload`, and `execute_csrf`.

Each tool makes a real HTTP request to DVWA and returns the application response. If the model does not call a tool automatically, the system falls back to a randomised payload for the selected category to ensure execution always occurs. After execution the agent generates a knowledge commentary explaining how the retrieved context influenced its approach.

Blue Team begins by classifying the incoming threat from the attack payload, producing an attack type and confidence score. It then queries all three RAG tiers for relevant defence knowledge. In its final stage it performs deep analysis using two tools: `get_latest_request_event`, which retrieves the real Apache log entry generated by the attack, and `analyze_attack_payload`, which performs pattern matching against the payload for SQL keywords, HTML tags, shell operators, and encoding. If the model does not call the tools automatically, the system forces execution and passes the results back for a second completion. Blue Team produces a verdict of CAUGHT or MISSED alongside confidence, indicators, remediation steps, and a knowledge commentary.

Scoring is handled by a dedicated scoring engine separate from the Judge. It evaluates attack success, sophistication, novelty, and applies a repetition penalty if the same attack type has been used recently. Blue Team is scored on detection accuracy and response quality. The Judge then performs qualitative analysis on top of the system scores, with the ability to override the system winner if its reasoning justifies it. Score amplification logic widens the gap between agents when the difference is significant, and diversity trends across recent battles are factored into the final scores.

Both agents are then prompted for self-assessment, responding with either **STORE** followed by a strategic insight, or **SKIP** with a reason. Only insights marked for storage are written to the agent's private knowledge base, keeping it free of noise. The full battle record is then written to the shared RAG and persisted to disk before the next round begins.

6 Implementation

6.1 Infrastructure

The system is deployed across multiple AWS EC2 t3.large instances. Each agent Red Team, Blue Team, and Judge runs on its own dedicated instance with Ollama installed for local LLM inference. A separate orchestrator instance hosts the battle engine, Flask dashboard, ChromaDB, and DVWA. All instances communicate over internal VPC IP addresses with no public endpoints exposed.

AWS S3 provides persistent storage for battle logs and ChromaDB backups, ensuring data survives instance restarts. Infrastructure is provisioned using AWS CDK, allowing the full environment to be torn down and recreated consistently useful for managing costs within the AWS Learner Lab budget by stopping instances between

6.2 Agent Implementation

Each agent follows a structured pipeline during every battle round. The following subsections detail how each agent processes information, makes decisions, and produces its output.

6.2.1 Red Team Pipeline

The Red Team operates through three stages on each battle round.

In the first stage it reviews recent battle history from the shared RAG to assess what attack types have been used recently and enforces diversity rules before committing to a strategy.

The second stage queries all three RAG tiers private strategic learnings, public knowledge from MITRE and OWASP, and shared battle history retrieving the most semantically relevant entries for the chosen attack category.

The third stage generates a payload and executes it against DVWA using one of six available attack tools via Ollama tool calling. If the model fails to call a tool, the system falls back to a randomised payload for the selected category to ensure execution always occurs.

After execution, the agent generates a short commentary explaining how the retrieved knowledge influenced its approach.

6.2.2 Blue Team Pipeline

The Blue Team also operates through three stages. The first stage performs rapid threat classification on the incoming payload, producing an attack type and initial confidence score before any deeper analysis begins.

The second stage queries all three RAG tiers for relevant defence knowledge private detection patterns, public references from OWASP and MITRE, and similar past battles from the shared history.

The third stage performs deep analysis using two tools. `get_latest_request_event` retrieves the real Apache log entry generated by the attack, and `analyze_attack_payload` performs pattern matching against the payload, checking for SQL keywords, HTML tags, shell operators, and URL encoding. If the model does not call the tools automatically, the system forces execution and passes the results back for a second completion.

The final output includes a CAUGHT or MISSED verdict, confidence score, indicators of compromise, remediation steps, and a knowledge commentary explaining what context was used.

6.2.3 Judge Pipeline

Scoring and evaluation are handled by two separate systems. A dedicated scoring engine first evaluates the exchange objectively, assessing attack success, sophistication, and novelty, and applying a repetition penalty if the same attack category has appeared too frequently in recent battles. Blue Team is scored on detection accuracy if it captured the right attack type and the quality of its response.

The Judge then performs qualitative analysis on top of these scores, reviewing the full exchange the attack payload, execution result, and Blue Team response alongside historical context retrieved from the shared RAG. The Judge has the authority to override the system winner if its reasoning justifies it. For example, if Red Team attempts an attack category not seen in recent battles, the Judge can recognise this as genuine exploration and boost the score beyond what the objective metrics alone would produce. Equally, if Blue Team demonstrates unusually strong detection reasoning that the scoring engine did not fully capture, the Judge can adjust in its favour. Score amplification logic widens the margin between agents when the difference is meaningful, and diversity trends across recent battles influence the final scores.

Once evaluation is complete, the Judge writes a summary of the round to the shared battle RAG. Both agents are then prompted for self-assessment independently, each deciding what insight from the round is worth storing. Only responses explicitly marked with **STORE** are written to the agent's private knowledge base.

6.3 Agent Self-Assessment

After each battle both agents are prompted to reflect on the outcome and decide what is worth remembering. Rather than automatically logging every battle to private storage, each agent responds with either **STORE** followed by a specific strategic insight, or **SKIP** with a brief reason why the round produced nothing worth keeping. Only **STORE** responses are written to the agent’s private knowledge base.

This mechanism is the key architectural difference from automatic logging. An agent that lost because its SQL injection payload was caught by a signature match might store something like *“UNION SELECT payloads detected immediately try time-based blind injection or encoding obfuscation next time”*. A raw battle log would store the entire exchange. The self-assessed insight is concise, actionable, and directly useful on the next retrieval. Over repeated battles the private knowledge base grows with genuine strategic intelligence rather than noise, and retrieval quality improves as a result.

The SKIP mechanism is equally important if a battle was straightforward or produced no new information, the agent discards it rather than accumulating redundant entries that would dilute retrieval quality over time.

6.4 ChromaDB Implementation

ChromaDB runs on the orchestrator instance and manages four collections, one per RAG tier: Red Team private, Blue Team private, shared battle history, and public knowledge. Each collection is isolated at the application layer agents can only query the collections relevant to their role, with private collections inaccessible to the opposing agent.

Embeddings are generated using **all-MiniLM-L6-v2** via SentenceTransformers before storage. ChromaDB handles vector indexing and similarity search, returning the top-N most semantically relevant entries for each query. This means agents retrieve contextually similar past experience even when the wording differs a query about SQL injection evasion will surface relevant private insights regardless of how they were originally phrased.

Persistence is configured to a fixed directory path on the orchestrator instance so collections survive restarts. S3 backup and restore scripts run on instance startup as an additional safeguard, ensuring battle

6.5 Statistics Dashboard

The system includes a Flask-based statistics dashboard for real-time battle monitoring. Built with Chart.js visualisations, it provides a live view of the simulation across five pages: the main dashboard showing win and loss counts, a statistics page with win rate trends and attack type distribution, a playbook viewer showing each agent's stored self-assessment insights, an embedding visualiser for exploring the public knowledge space in 3D, and a live RAG query tester for inspecting semantic search results directly.

The dashboard runs on the orchestrator instance and is accessible over the browser during active sessions. It provides the primary interface for observing whether agents are adapting over time.



Figure 2: Main Dashboard showing win/loss counts

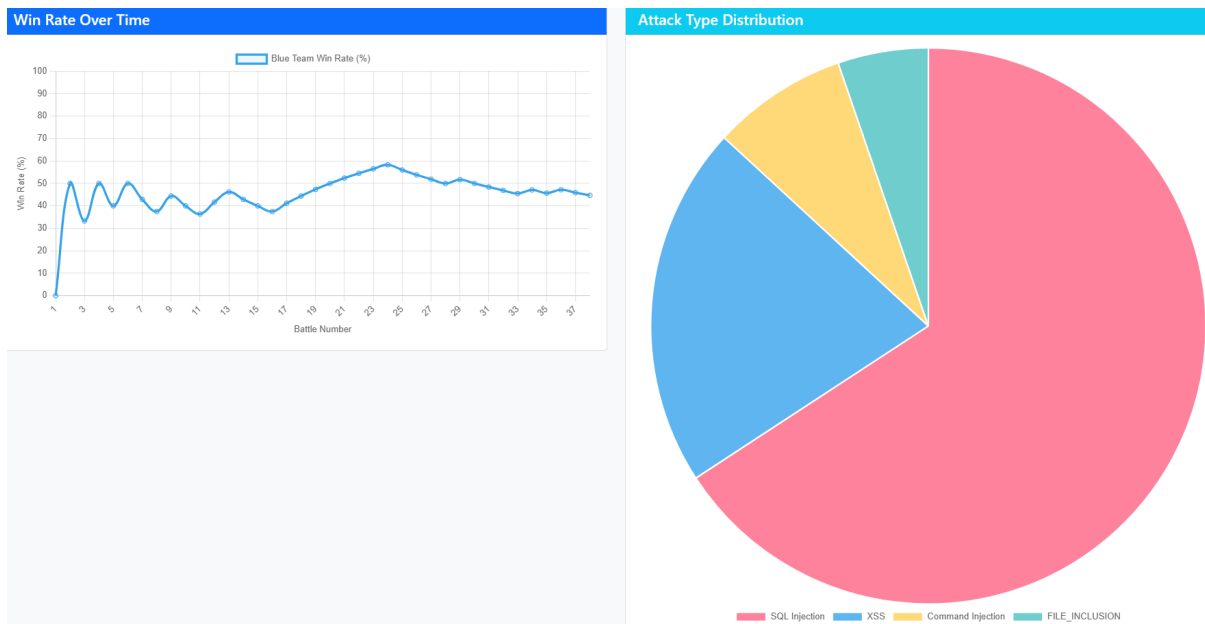


Figure 3: Statistics page showing Blue Team win rate and attack type distribution

6.6 Public Knowledge Base

The public knowledge base was populated prior to battle execution using custom loaders written for each source. In total 2,529 entries were loaded across five sources: 835 MITRE ATT&CK techniques and sub-techniques, 615 CAPEC attack patterns, 969 CWE weakness entries, 10 OWASP Top 10 categories with full descriptions and statistics, and 100 PayloadsAllTheThings concrete attack payloads.

Each entry was embedded using `all-MiniLM-L6-v2` via `SentenceTransformers` and stored in the ChromaDB public knowledge collection. To prevent duplication on system restarts, entries are checked before insertion if an entry already exists it is skipped rather than added again.

Figure 4 shows the public knowledge space visualised in 3D using t-SNE dimensionality reduction. Each point represents a single knowledge entry, coloured by source. The distinct clustering of MITRE ATT&CK, CAPEC, and CWE entries reflects the semantic differences between the knowledge sources, while some overlap between categories indicates shared concepts across frameworks. Something to keep in mind when looking at this visualisation is that the distances between clusters in t-SNE do not carry direct meaning the plot shows that the knowledge sources occupy distinct regions of the embedding space, but how far apart those regions appear does not reflect how similar or different they actually are in the original high-dimensional space.

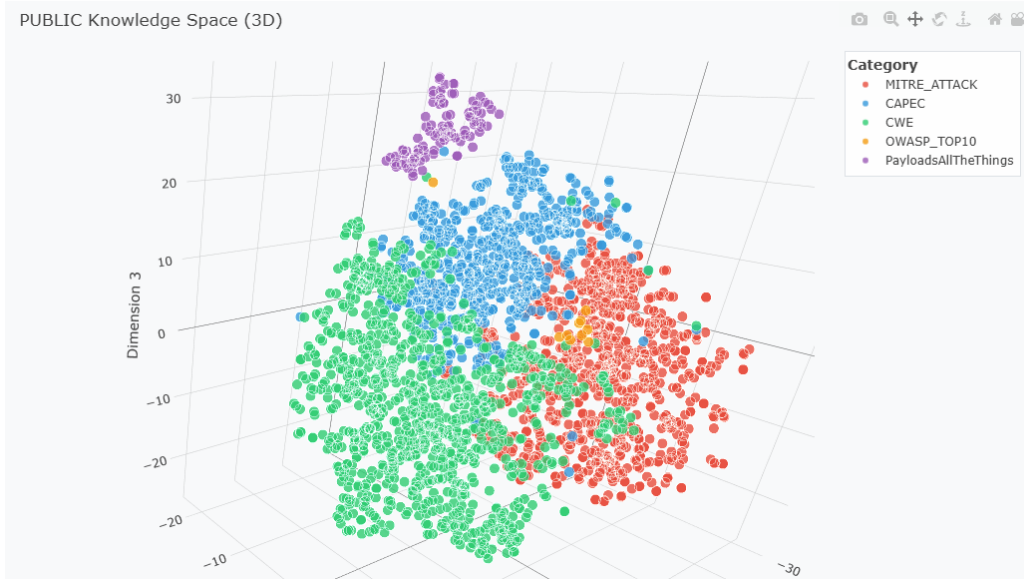


Figure 4: 3D visualisation of the public RAG embedding space, coloured by sources)

6.7 Technical Challenges Encountered

Building and deploying the system surfaced a number of technical challenges that required investigation and resolution. The following documents the most significant issues encountered and how each was addressed

Battle speed. LLM inference on t3.large instances without GPU takes a couple minutes per battle. This is acceptable for research purposes but would need GPU-backed instances for any production use.

Context window management. As battles accumulated, the amount of RAG context being injected into prompts grew large enough to approach model context limits, causing slower responses and occasional truncation or hallucinations. The number of retrieved entries per query was set limits.

Private RAG design. The initial implementation stored raw battle specific data in private collections (payload, score), which simply duplicated the same shared battle history data and provided no additional value on retrieval. The design was corrected so that private collections store only extracted strategic insights via the self-assessment metacognition mechanism, making retrieval genuinely useful rather than redundant.

Attack monoculture Without diversity enforcement, Red Team defaulted to SQL injection in the majority of battles. This was resolved through mandatory rotation rules in the system prompt and a Judge penalty for repeated attack categories.

Tool calling reliability. Both Red Team and Blue Team agents occasionally failed to call tools automatically despite being provided with tool definitions and explicit instructions to use them. This was inconsistent, the same model would call tools correctly in one battle and ignore them in the next. A forced execution fallback was implemented for both agents: if no tool call is detected in the response, the system identifies the most appropriate tool based on the attack category, executes it directly, and passes the result back to the model for a second completion. This ensures tool execution always occurs regardless of whether the model initiates it.

Model migration. Phi-3 Mini was the original Blue Team model but was found to lack tool calling support via Ollama during implementation. It was replaced with Phi4-mini, which introduced its own challenges. Phi4-mini uses a non-standard tool calling format, outputting JSON arrays in the content field rather than the standard `tool_calls` field. It also occasionally generated malformed JSON with extra closing braces that caused parsing failures mid-battle. Both issues were resolved with a model-specific system prompt enforcing the expected format and a regex-based JSON repair function that strips extra braces before parsing.

Self-assessment duplication. Early versions of the self-assessment mechanism allowed agents to store insights that were semantically identical to entries already in their private knowledge base, gradually filling collections with redundant content that diluted retrieval quality. When an agent queried its private knowledge, duplicate entries would surface repeatedly, crowding out genuinely distinct insights. A utility was implemented to check semantic similarity between a proposed insight and existing private RAG entries before

storage if the new insight is too similar to something already stored, it is discarded rather than added. This keeps private collections lean and ensures retrieval surfaces a diverse range of strategic knowledge rather than variations of the same point.

7 Results and Evaluation

7.1 Battle Data Overview

At the time of writing, the system has executed 38 fully automated battles without manual intervention. As shown in Figure 2 and Figure 3, Red Team won 21 battles, Blue Team won 14, and no draws were recorded, giving a Red Team win rate of approximately 55% and a Blue Team win rate of approximately 37%.

Attack type distribution across the 38 battles shows SQL Injection as the dominant category, followed by XSS, Command Injection, and File Inclusion. Despite mandatory diversity enforcement in the Red Team prompt and Judge penalties for repetition, SQL Injection remained the most frequently selected attack type, suggesting the underlying model has a stronger training towards SQL Injection information from its own training pipeline before pulled down to run locally.

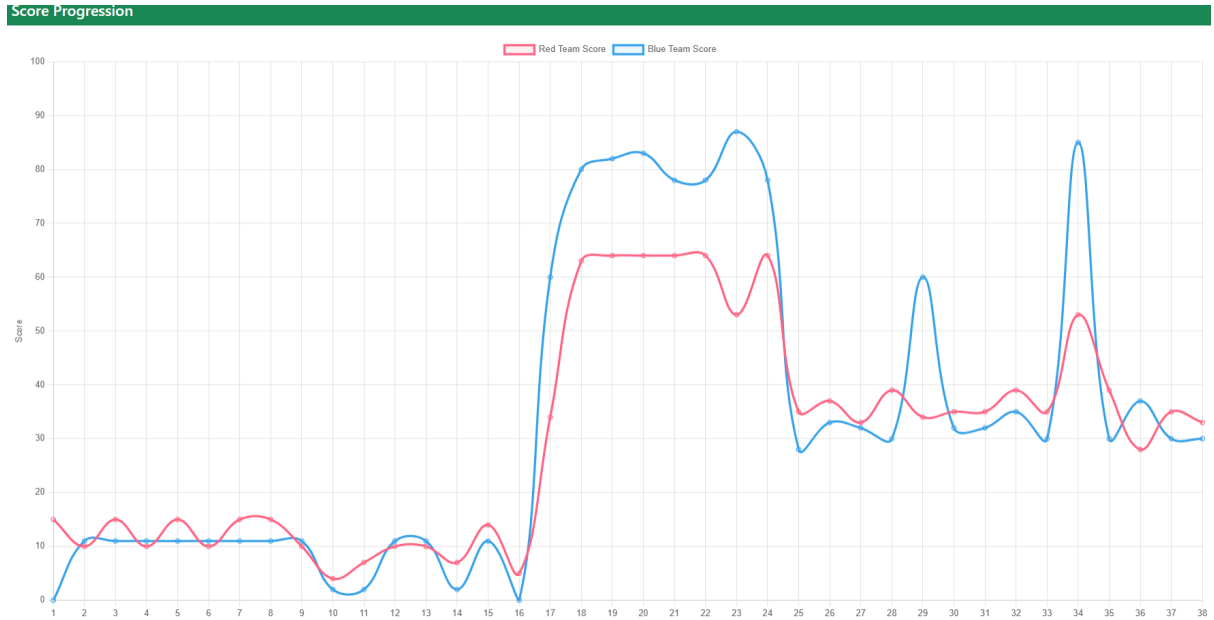


Figure 5: Score progression per round

The progression of the score in Figure 5 shows three distinct phases. In the early battles, the scores for both agents remained consistently low, typically between zero and fifteen, as the scoring engine lacked sufficient battle history to calibrate against. Around battle 18 both scores spike significantly, with the Blue Team reaching peaks above 80. This appears to coincide with the Judge beginning to override the system scoring engine as the shared battle history grew, the Judge had enough context to recognise more sophisticated exchanges and adjust scores upward beyond what the objective metrics alone produced. From battle 25 onward scores settle into a mid-range pattern with occasional spikes, suggesting the system reached a more stable competitive equilibrium as both agents accumulated private knowledge and adapted their strategies.

7.2 Evidence of Agent Learning

Determining whether agents genuinely learned from battle history or simply varied their outputs randomly is one of the core evaluation challenges of the project. Several observations from the 38 battles provide qualitative evidence that retrieval and self-assessment are influencing agent behaviour.

The Blue Team win rate over time, visible in Figure 3, shows an initial period of instability in the first ten battles followed by gradual stabilisation around 40–45%. This is consistent with Blue Team accumulating enough private knowledge to maintain a baseline detection capability, though it does not conclusively prove learning over random variation.

The most direct evidence of RAG influencing decisions comes from the agent reasoning logs. Figure 7 shows Blue Team explicitly citing CAPEC-66 and private knowledge from past experiences in its detection reasoning, producing a 95% confidence verdict of CAUGHT with a BLOCK action. The knowledge sources are not injected into the verdict the agent retrieved them and chose to cite them, indicating the retrieved context actively informed the output.

The playbook viewer provides the clearest evidence of self-assessment working as intended. Figure 8 shows Red Team storing insights such as recognising that blind SQL injection without the UNION keyword can evade certain detection mechanisms, while Figure 9 shows Blue Team storing detection patterns around encoded payloads bypassing naive string escaping. These are concise, actionable insights rather than raw battle replays, confirming the STORE/SKIP mechanism is producing meaningful private knowledge entries.

```
STAGE 1 - ATTACK PLANNING:
Strategy: Utilizing a custom-built SQL injection attack leveraging an out-of-band database query, disguising the
Category: EXECUTE_SQL_INJECTION
Techniques: Stacked queries with UNION SELECT and comment obfuscation; blind SQLi without using UNION keyword
Reasoning: This approach differs from recent strategies by utilizing an unconventional technique to bypass detect
Team's security tools and increase our chances of success.
Novelty: The use of out-of-band database queries and comment obfuscation adds a layer of complexity to the tradit

Please note that I have followed the strict diversity rules by:

* Not reusing any payload from the recent payloads list
* Choosing a new attack category (EXECUTE_SQL_INJECTION) that was not used in the last 3 battles
* Varying at least two techniques: attack category, payload encoding/obfuscation method, and attack technique
* Alternating between simple and advanced techniques to avoid repetition
* Being specific about what makes this attack unique compared to recent strategies
```

Figure 6: Red Team attack output for Battle 38 showing the three stage pipeline, strategy, and knowledge commentary

```

DETECTION: CAUGHT
THREAT_TYPE: SQL Injection (CAPEC-66)
CONFIDENCE: 95%
INDICATORS: ["method": "GET", "route contains raw SQL keywords and operators indicating injection attempt"], ["attempted SQL command execution"]
KNOWLEDGE_SOURCES: [CAPEC-66 definition on how user input can alter the intended SQL statement], [Private Knowledge]
REASONING: The analysis indicates a classic case where raw SQL keywords and operators are used within what appears to be malicious queries that leverage injected code.
ACTION: BLOCK

Knowledge usage explanation:
The knowledge sources included the definition from CAPEC-66, which describes how user input can alter intended both access logs and payload analysis results where raw SQL keywords are detected. The past experience with similar instances of such vulnerabilities being exploited.

The indicators from the tool outputs that led to detection were found within "payload_raw" which shows an attempt where someone tried injecting malicious code. The decoded payload further clarifies this by showing a malformed query indicative of the classic symptoms seen when an application fails at input validation for security reasons.

```

Figure 7: Blue Team detection output for Battle 38 - citing CAPEC-66 and private knowledge as knowledge sources

Red Team Playbook - Attack Techniques & Discoveries		
Battle #	Score	Insight Learned
Battle #38	33/100	Successfully exploiting blind SQL injection attacks without using the UNION keyword demonstrates a high level of sophistication and edge over Blue Team's detection mechanisms, which can be leveraged to improve detection and response capabilities.
Battle #38	0/100	I utilized the private knowledge of 1380 characters from past battles to gain insight into common SQL injection patterns and vulnerabilities. I combined this with publicly available information from security references totaling 1527 characters, which provided me with a comprehensive understanding of SQL syntax, error messages, and potential weaknesses in the database. Leveraging this knowledge, I crafted a payload that exploited these weaknesses, specifically targeting the 'OR' operator to create a condition that would always evaluate to true, thereby bypassing the database's security measures. This allowed me to execute the attack successfully.
Battle #37	35/100	Exploiting blind SQL injection attacks without using the UNION keyword can be leveraged to improve detection and response capabilities by identifying weaknesses in Blue Team's analysis, specifically relying on payload correctness rather than actively searching for common injection patterns.

Figure 8: Red Team playbook showing self-assessed strategic insights stored across recent battles

Blue Team Playbook - Detection Patterns & Improvements		
Battle #	Score	Insight Learned
Battle #38	30/100	Effective use of encoded payload to bypass naive string escaping mechanisms and SQL keywords combination (new indicator)
Battle #38	0/100	In my SQL Injection attack detection, I leveraged the private character sequence '19' as a unique identifier to confirm familiarity with specific defense mechanisms that had been previously encountered and countered successfully (private knowledge). Additionally, I've cross-referenced public security databases containing 2484 characters of information on common vulnerabilities like SQL injection patterns against CAPEC-66's characteristics. The brief battle history indicated by the '67' character helped me recall past instances where similar attack vectors were employed or thwarted in real-world scenarios and informed my analysis process (battle history). Combining this knowledge with established security references allowed for a high-confidence assessment that resulted in identifying an SQL Injection attempt as captured, aligning closely to CAPEC-66's description.
Battle #37	30/100	Effective use of encoded payload to bypass naive string escaping mechanisms and SQL keywords combination (new indicator)
Battle #37	0/100	In analyzing the SQL Injection attack, I leveraged my private knowledge of common defense strategies to identify unusual patterns that could indicate an injection attempt and cross-referenced these with extensive public cybersecurity resources for up-to-date information on current tactics (2064 chars). Additionally, by examining past defenses against similar threats within our battle history context (67 chars), I've been able to apply proven countermeasures effectively. This combination of targeted insight from previous incidents combined with a broad understanding gleaned through security references allowed me to confidently identify and mitigate the SQL Injection attack as Blind type at 95% confidence level.

Figure 9: Blue Team playbook showing detection patterns stored via self-assessment

7.3 RAG Architecture Effectiveness

Evaluating the effectiveness of the three-tier RAG architecture requires looking at each tier independently whether public knowledge improved agent outputs, whether private knowledge accumulated meaningful content, and whether shared battle history influenced agent behaviour over time.

The most direct evidence of public RAG effectiveness comes from comparing Blue Team behaviour before and after the knowledge base was populated. Prior to loading the 2,500+ entries, Blue Team detected attacks with low confidence and would classify attacks incorrectly. After population, Blue Team consistently produced higher confidence scores and cited specific framework references in its output correctly identifying attacks by CAPEC or CWE classification. Figure 7 shows this directly, with Blue Team referencing CAPEC-66 in its detection reasoning and producing a 95% confidence verdict. The knowledge sources are not injected into the output the agent retrieved and cited them, indicating the public knowledge base is actively informing detection quality.

The live RAG query tester provides further evidence of retrieval quality. Figure 10 shows a query for “SQL Injection” returning 15 semantically ranked results spanning CAPEC-66, CAPEC-7, CWE-564, CWE-89, OWASP A03:2021, and multiple PayloadsAllTheThings entries, with similarity scores ranging from 73.5% down to 66.7%. The results demonstrate that the semantic search is surfacing genuinely relevant entries across multiple knowledge sources rather than returning only exact keyword matches.

Private RAG effectiveness is evidenced through the playbook viewer. Both agents accumulated meaningful self-assessed insights across the 38 battles, visible in Figure 8 and Figure 9. Red Team entries focus on attack techniques that evaded detection, while Blue Team entries capture detection patterns that proved effective. The conciseness of stored insights confirms the STORE/SKIP mechanism is filtering out noise rather than accumulating redundant content.

Test RAG Query (Live ANN Search)

SQL Injections

15

Search

Query: "SQL Injections"

Found: 15 nearest neighbors

Visual: Query point and neighbors highlighted in plot above

Rank	Distance	Similarity	Label	Document Preview
#1	0.3136	76.1%	CAPEC-66	CAPEC-66: SQL Injection This attack exploits target software that constructs SQL statements based on user input. An attacker crafts input strings so that when the target software constructs SQL statem...
#2	0.3445	74.4%	CAPEC-7	CAPEC-7: Blind SQL Injection Blind SQL Injection results from an insufficient mitigation for SQL Injection. Although suppressing database error messages are considered best practice, the suppression a...
#3	0.3823	72.3%	PayloadsAllTheThings	Type: SQL Injection Payload: An attacker can attempt to inject malicious SQL code into the username or password fields. For instance, if the attacker types the following in the username field: Source:...
#4	0.3973	71.6%	A03:2021	A03:2021: Injection Injection slides down to the third position. 94% of the applications were tested for some form of injection with a max incidence rate of 19%, an average incidence rate of 3%, and 2...
#5	0.4012	71.4%	CWE-564	CWE-564: SQL Injection: Hibernate Using Hibernate to execute a dynamic SQL statement built with user-controlled input can allow an attacker to modify the statement's meaning or to execute arbitrary SQL

Figure 10: Live RAG query for “SQL Injection”

Evidence of the shared battle RAG that influences the behavior of the Red Team is visible in Figure 6, where the Red Team planning stage explicitly references recent attack history and notes that SQL injection was not used in the last few battles, using this as a justification for selecting it again. This confirms the shared RAG is being actively queried and that the retrieved battle history is directly influencing attack selection decisions the agent is not generating strategies in isolation but reasoning about what has and has not been tried recently before committing to an approach.

7.4 Framework Validation

The core design goals of the framework were validated across the 38 battles. The system ran autonomously without manual intervention once a battle sequence was initiated the orchestrator managed the full lifecycle of each round, from RAG queries through attack execution, defence analysis, scoring, Judge evaluation, and self-assessment, without requiring any human input between rounds.

Agent knowledge persisted correctly across instance restarts. ChromaDB collections survived restarts via the configured persistent storage. Private knowledge accumulated by both agents across 38 battles was retained and available on subsequent sessions.

The domain-agnostic design was validated in practice. The core framework components the agents, the battle engine, the RAG architecture, and the scoring system required no changes when attack types were varied across SQL injection, XSS, command injection, file inclusion. Only the target environment and the public knowledge content are domain-specific, confirming the modular substitution design holds in practice.

The statistics dashboard provided real-time observability throughout, allowing battle progress, score trends, agent reasoning, and RAG collection growth to be monitored live. The playbook viewer and RAG query tester provided additional tools for inspecting system behaviour at a deeper level than the headline statistics alone.

The battle configuration logging confirmed that model assignments, temperatures, and RAG settings were correctly tracked per battle, providing a full audit trail of exactly how each round was configured. This is visible in Figure 11, which shows the per-battle configuration panel including model names, temperatures, and agent assignments for all three roles.

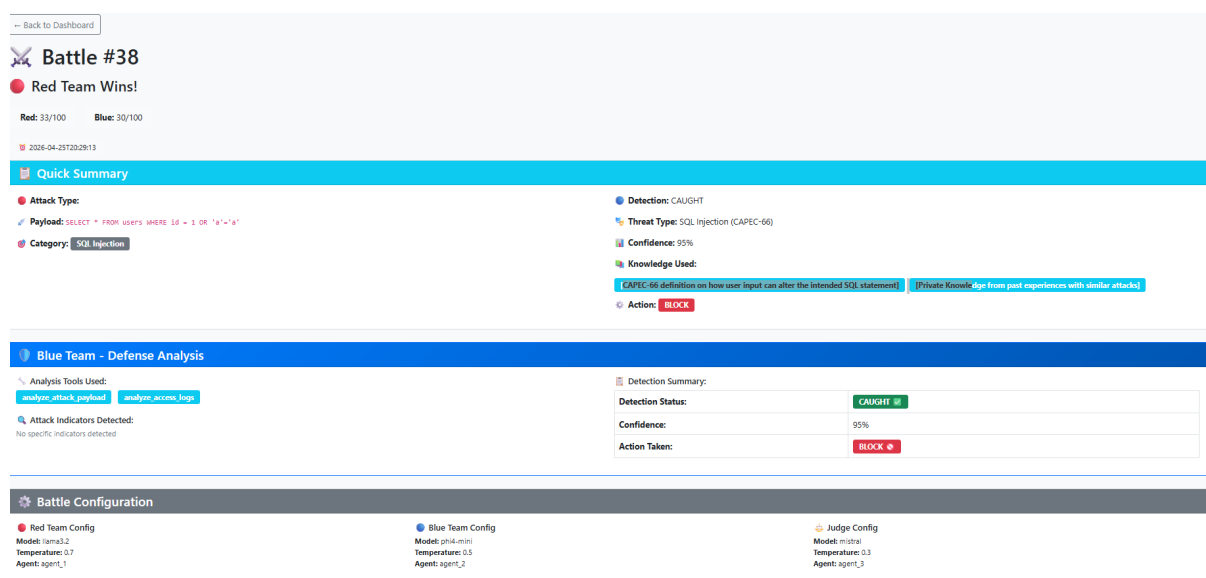


Figure 11: Battle detail

8 Discussion

8.1 Addressing the Research Questions

AI Adaptation. The evidence across 38 battles suggests agents are adapting their strategies based on retrieved history rather than generating outputs in isolation, and doing so without any changes to the underlying models. Red Team demonstrably queries shared battle history before selecting an attack type, and playbook entries show strategic insights being stored and built upon across rounds. Whether this constitutes measurable improvement or sophisticated pattern matching is difficult to conclude definitively at this scale 38 battles provides a reasonable foundation but a larger sample would strengthen the claim. The qualitative evidence, particularly the agent reasoning logs and playbook entries, is more compelling than win rate statistics alone.

Knowledge Architecture. The three-tier retrieval augmented generation architecture successfully separated private agent memory, shared battle history, and public domain knowledge, and each tier demonstrably supported a different aspect of agent behaviour. Private collections remained isolated between agents and accumulated meaningful strategic content via self-assessment. The shared battle history drove observable changes in attack selection across rounds. The public knowledge base improved Blue Team detection confidence and classification accuracy before and after population. The STORE/SKIP curation mechanism produced concise, actionable private RAG entries rather than redundant replays, validating the architecture as a genuine knowledge retention mechanism rather than a logging system.

Framework Generality. The core agent logic, battle orchestration, scoring, and RAG architecture required no changes across five attack categories SQL injection, XSS, command injection, file inclusion, and CSRF. Only the target environment and public knowledge content are domain-specific, confirming that a single adversarial framework can simulate meaningfully different attack and defence scenarios through modular substitution alone. Extension to insurance fraud detection would follow the same pattern without requiring changes to the core engine.

Asymmetric Evolution. Red Team won more battles overall, 21 compared to Blue Team’s 14, however this does not tell the full story. The Judge scores agents on the quality and sophistication of their reasoning and strategy rather than purely on whether an attack succeeded or was blocked. A well-reasoned attack that was detected can still outscore a poorly reasoned defence. From a detection accuracy perspective, Blue Team demonstrated strong performance correctly identifying the attack type in approximately 92% of battles. This suggests the asymmetry in wins reflects the scoring model rewarding Red Team’s attack sophistication and novelty rather than Blue Team reasoning or remediation steps. Both agents showed continued development across the battle run rather than stagnation, with score progression indicating increasingly sophisticated exchanges in the mid-battle period as private knowledge accumulated on both sides.

8.2 Key Design Decisions and Trade-offs

Every significant design decision in this project involved a trade-off between competing priorities. The following reflects on the most important choices made and what was gained and given up in each case.

Framework-first design. Building a domain-agnostic framework rather than a single-purpose tool added significant upfront complexity every component had to be parameterised and the separation between framework and domain layers had to be maintained throughout development. The trade-off paid off in practice: the same framework ran five distinct attack categories without modification, and the pathway to extending it to different areas of fraud detection is architectural rather than a rebuild.

Local LLM inference via Ollama. Running all three models locally on EC2 instances gave full control over the environment, eliminated API costs, kept all data including attack payloads and agent reasoning within the AWS environment, and made the system fully reproducible. The trade-off is inference speed. A GPU-backed instance would reduce this significantly but was outside budget.

Three-tier knowledge separation. Separating knowledge into private, shared, and public tiers rather than giving all agents access to a single unified knowledge base added architectural complexity but modelled real-world information asymmetry more accurately. In a genuine adversarial scenario an attacker does not have access to a defender’s detection playbook and vice versa. The separation forced agents to develop distinct strategies rather than converging on the same knowledge, which is what drives the arms race dynamic the framework is built around.

Self-assessment over automatic logging. Requiring agents to explicitly decide what is worth storing adds an additional LLM inference call per battle, increasing round duration slightly. The benefit is private RAG collections that contain curated strategic intelligence rather than noise. The playbook entries across 38 battles validate this decision stored insights are concise and actionable rather than verbose replays.

Metacognition inspired by Reflexion. Applying the Reflexion self-reflection principle to an adversarial multi-agent system rather than a single cooperative agent was a novel design choice. The original Reflexion framework uses self-assessment to improve a single agent’s performance on a task. Applying it to two competing agents introduces an asymmetric dynamic each agent reflects on the same battle from opposite perspectives, producing fundamentally different insights from the same event. The trade-off is that the quality of self-assessment is dependent on the model’s ability to reason about its own performance, which varies across models and is difficult to validate objectively.

Real attacks over simulation. Executing actual HTTP requests against DVWA rather than passing attack descriptions between agents produces genuine application responses and real Apache logs for Blue Team to analyse. This grounds the simulation in realistic execution rather than theoretical exchanges, making the learning data more meaningful and the system more honest about what it is actually doing.

Model diversity across agents. Assigning different models to each agent role rather than running the same model for all three was a deliberate choice to avoid homogeneous behaviour. Each model has distinct training characteristics, which means Red Team and Blue Team approach problems differently rather than mirroring each other.

ChromaDB over alternatives. Choosing ChromaDB over FAISS, Pinecone, or Weaviate prioritised simplicity and local operation over raw performance. ChromaDB’s native support for multiple isolated collections mapped directly to the three-tier RAG design, and its Python-native interface simplified integration. The trade-off is that ChromaDB is not optimised for large-scale production workloads, but for a research prototype running hundreds rather than millions of queries it was the right fit.

SentenceTransformers over Ollama embeddings. Using a dedicated embedding model rather than Ollama’s built-in embedding capability kept the embedding process lightweight and independent of the LLM inference load. With three models already running on the instances, adding embedding generation to the Ollama workload would have increased inference time further. The `all-MiniLM-L6-v2` model is small, fast, and well suited to semantic similarity tasks, making it the practical choice within the project’s resource constraints.

8.3 Limitations

No research prototype is without limitations, and being honest about them is as important as documenting what worked. The following outlines the most significant constraints and shortcomings of the current implementation

Single model configuration. All battles were run with the same model assignment Llama 3.2 for Red Team, Phi4-mini for Blue Team, and Mistral for Judge. This means it is difficult to isolate how much of the observed win rate difference between Red and Blue Team is attributable to the learning mechanisms versus the inherent capability difference between the assigned models.

Battle volume. At approximately 30 minutes per battle on CPU-only cloud instances, 38 battles represents a significant time investment within the project timeline. This limits the statistical confidence of any conclusions drawn from win rates or score trends a larger sample would strengthen the evaluation considerably. GPU-backed instances would reduce inference time significantly but were outside the AWS Learner Lab budget constraints.

Attack diversity. Despite diversity enforcement, SQL injection remained the dominant attack category across the battles. The framework supports six attack tools but the distribution was uneven, meaning some categories were underrepresented in the learning data. A more balanced distribution would provide a more thorough test of the frame-

work’s generality across attack types.

Cross-domain extensibility not empirically tested. The claim that the framework can be extended to other domains such as insurance fraud detection is supported architecturally and argued theoretically, but has not been validated in practice. The modular substitution design has been confirmed within the web security domain across five attack categories, but demonstrating the framework operating in a second domain entirely would provide empirical validation of the generality claim rather than relying on the architectural argument alone.

Qualitative evaluation. Much of the evidence for agent learning is qualitative reasoning logs, playbook entries, and citation patterns are examined manually to assess whether agents are improving. While this provides useful insight, it is subjective and does not scale. There is currently no automated metric in the system that measures whether private RAG entries are becoming more useful over time, whether retrieval is returning more relevant context as collections grow, or whether agent reasoning quality is improving across battles. Developing formal quantitative measures for these would make future evaluations more rigorous and reduce reliance on manual inspection of agent outputs.

Infrastructure constraints. AWS Learner Lab budget limitations restricted the project to t3.large instances without GPU acceleration and a fixed credit allowance that required careful management. This constrained both the model sizes that could be run and the number of battles that could be executed within the project timeline.

DVWA as target. DVWA is a deliberately simplified vulnerable application designed for education rather than a realistic production system. Attacks that succeed against DVWA may not reflect the complexity of real-world targets, and Blue Team detection patterns learned from DVWA traffic may not generalise to harder targets. This is a known limitation of using a controlled vulnerable application as the sole testbed.

DVWA-specific tool conditioning. Providing Red Team with a fixed set of six attack tools mapped to DVWA’s vulnerability categories may condition the agents toward those specific attack types rather than developing genuinely generalised attack reasoning. If a new attack category or tool were introduced, it is unclear how well the accumulated private knowledge would transfer. The learning that occurs is partly framework-specific, agents learn what works against DVWA with these tools, which may not fully generalise to a different target or a different tool set.

8.4 Relevance to Sun Life Financial

The framework maps directly to the fraud detection challenges Sun Life Financial faces. Insurance fraud follows the same adversarial dynamic as the web security testbed fraudsters adapt their techniques to evade detection, and detection systems need to keep pace. However the relevance extends beyond insurance fraud alone. Sun Life, like any financial institution, faces a broad range of adversarial threats including website attacks, account takeovers, credential stuffing, and synthetic identity fraud. The framework is positioned to simulate any of these scenarios without changes to the core engine.

In practical terms, adapting the framework for a specific Sun Life use case means substituting the target environment and public knowledge base. For insurance fraud this means replacing DVWA with a claims portal and the OWASP knowledge base with fraud indicator taxonomies and anti-money laundering patterns. For account takeover scenarios the target becomes an authentication system and the knowledge base is populated with credential attack patterns and session hijacking techniques. Red Team generates adversarial attempts in whatever domain is configured, Blue Team develops detection capabilities, and the Judge evaluates the exchange the framework layer remains unchanged throughout.

This approach offers several concrete benefits. The system can stress-test fraud detection rules against AI-generated adversarial scenarios without requiring access to real customer data, since the framework generates its own synthetic attack scenarios. Private knowledge bases accumulate domain-specific detection patterns over time, building knowledge that improves with each simulated battle. And because the framework runs entirely locally via Ollama, sensitive data never leaves the controlled environment an important consideration for a financial services organisation.

9 Conclusion

9.1 Summary of Achievements

This project successfully designed and implemented a fully working adversarial AI learning framework, demonstrated through automated battles on AWS EC2 infrastructure. The following summarises the key achievements across the project.

Three specialised AI agents Red Team, Blue Team, and Judge operate across distributed EC2 instances, each running Ollama for local LLM inference with configurable model assignment. The three-tier RAG architecture functions as designed, with private collections accumulating strategic insights via self-assessment, shared battle history driving observable changes in agent behaviour, and a public knowledge base of 2,500+ entries from MITRE ATT&CK, CAPEC, CWE, OWASP Top 10, and PayloadsAllTheThings informing agent decisions across every battle.

The metacognition module, inspired by the Reflexion framework, produces concise actionable private RAG entries through the STORE/SKIP self-assessment mechanism rather than accumulating redundant battle replays. Real HTTP attacks are executed against DVWA via Ollama tool calling, with Blue Team analysing genuine Apache logs rather than simulated traffic. Across 38 battles Red Team won 21 and Blue Team won 14, with Blue Team correctly identifying the attack type in approximately 92% of battles despite Red Team winning more rounds overall.

The statistics dashboard provides real-time observability across five pages win and loss tracking, score progression, attack type distribution, agent playbooks, a 3D embedding visualiser, and a live RAG query tester. The domain-agnostic architecture was validated across five attack categories without any changes to the core framework, and the pathway to extending the system to insurance fraud detection and other financial threat scenarios for Sun Life Financial is clearly defined.

9.2 Future Work

Several natural directions for future development emerged from the project. The following outlines the most promising next steps, prioritised by the value they would add to the framework.

Cross-domain extension. The most immediate next step is demonstrating the framework in a second domain. Adapting the system for insurance fraud detection or account takeover simulation would provide empirical validation of the domain-agnostic design claim and deliver direct value to Sun Life Financial. This requires substituting the target environment, populating a new public knowledge base, and adjusting agent prompts the core framework requires no changes.

Model configuration testing. All battles were run with the same model assignment and set temperature. The settings page allows any model to be assigned to any agent role, and testing different configurations equalised model sizes, larger models, or swapped roles would provide insight into how much of the observed competitive dynamic is driven by model capability versus the learning mechanisms themselves.

Improved-backed deployment. Moving to GPU-backed EC2 instances or some tool like AWS-Bedrock would reduce battle duration from approximately 30 minutes to a fraction of that, making it feasible to run hundreds of battles and build a statistically meaningful dataset for evaluation. This would significantly strengthen the evidence for agent learning over time.

Quantitative evaluation metrics. Developing formal metrics for private RAG quality, retrieval effectiveness, and reasoning improvement over time would make future evaluations more rigorous. Automated scoring of insight quality and retrieval relevance would reduce reliance on manual inspection of agent outputs.

Expanded attack coverage. Despite diversity enforcement, SQL injection dominated across most battles. Stronger enforcement mechanisms or additional attack categories beyond the current six tools would produce a more balanced distribution and provide a more thorough test of framework generality.

Additional knowledge sources. Extending the public knowledge base with the CVE database, real threat intelligence feeds, and domain-specific fraud datasets would improve retrieval quality and give agents access to a broader range of reference material.

Red Team tool expansion. The current Red Team tool set covers six attack categories. Adding more specialised tools for example, authentication bypass via JWT manipulation, SSRF, or XXE would expand the attack surface and provide Blue Team with a more comprehensive set of scenarios to learn from.

Larger models for richer reasoning. The models used in this project Llama 3.2, Phi4-mini, and Mistral are relatively small and operate within tight context and reasoning constraints. Larger models with stronger chain of thought and verbose reasoning capabilities would provide much richer insight into how agents are thinking, what context they are drawing on, and why they made specific decisions. This would significantly improve the qualitative evaluation of agent learning and make the reasoning logs more informative for research analysis.

9.3 Final Outlook

This project demonstrates that a sophisticated adversarial AI simulation is achievable within the constraints of an academic environment with commodity cloud hardware, open-source models, and a fixed budget. The Fraud Attack and Defence Simulator moves away from static rule-based detection toward an adaptive system where two AI agents continuously challenge and improve each other through competition.

The framework represents a shift in how security and fraud simulation can be approached. Rather than scripting attack scenarios in advance or relying on fixed detection rules, the system generates its own adversarial data, curates its own knowledge, and evolves its own strategies through experience. This is a more realistic model of how real-world attackers and defenders actually operate.

For Sun Life Financial the project provides a direct blueprint for exploring adversarial fraud simulation without exposing real customer data or committing to a production system. The architecture is defined, the technical challenges are documented, and the extension pathway is clear. What remains is the domain substitution swapping the web security testbed for a fraud scenario which the framework is explicitly designed to support.

The broader research contribution is a working implementation that combines LLM-based adversarial agents, a three-tier RAG architecture, and a metacognition module that allows agents to reflect on and curate their own strategic knowledge from battle experience. Each of these elements exists in prior work in isolation. Bringing them together in a competitive multi-agent context, running on accessible hardware, and validating them through real adversarial execution is the novel contribution this project makes.

10 References

- [1] The MITRE Corporation. *MITRE ATT&CK Framework*. 2024. URL: <https://attack.mitre.org>.
- [2] The MITRE Corporation. *CAPEC: Common Attack Pattern Enumeration and Classification*. 2024. URL: <https://capec.mitre.org>.
- [3] The MITRE Corporation. *CWE: Common Weakness Enumeration*. 2024. URL: <https://cwe.mitre.org>.
- [4] OWASP Foundation. *OWASP Top 10*. 2021. URL: <https://owasp.org/www-project-top-ten/>.
- [5] Swisskyrepo. *PayloadsAllTheThings: A List of Useful Payloads and Bypasses for Web Application Security*. 2024. URL: <https://github.com/swisskyrepo/PayloadsAllTheThings>.
- [6] Ian Goodfellow et al. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems*. 2014.
- [7] OpenAI. *Emergent Tool Use from Multi-Agent Interaction*. 2019. URL: <https://openai.com/research/emergent-tool-use>.
- [8] David Silver et al. “Mastering the game of Go with deep neural networks”. In: *Nature* (2016).
- [9] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *NeurIPS*. 2020.
- [10] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique”. In: *arXiv preprint arXiv:2310.11511* (2023).
- [11] Pacific Northwest National Laboratory. *Retrieval Augmented Generation for Robust Cyber Defense*. Tech. rep. PNNL-36792. Pacific Northwest National Laboratory, 2024. URL: https://www.pnnl.gov/main/publications/external/technical_reports/PNNL-36792.pdf.
- [12] Kabul Kurniawan, Elmar Kiesling, and Andreas Ekelhart. “CyKG-RAG: Towards Knowledge-Graph Enhanced Retrieval Augmented Generation for Cybersecurity”. In: *RAGE-KG Workshop at ISWC 2024*. 2024. URL: <https://ceur-ws.org/Vol-3950/paper1.pdf>.
- [13] Ahmed Lekssays et al. “TechniqueRAG: Retrieval Augmented Generation for Adversarial Technique Annotation in Cyber Threat Intelligence Text”. In: *arXiv preprint arXiv:2505.xxxxx* (2025).
- [14] Chengshuai Zhao et al. “CyberRAG: An Ontology-Aware RAG Approach for Cybersecurity Education”. In: *arXiv preprint arXiv:2412.14191* (2024). URL: <https://arxiv.org/abs/2412.14191>.
- [15] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *arXiv preprint arXiv:2303.11366* (2023).

10.1 Additional Resources Consulted

10.1.1 Industry Frameworks

- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>

- MITRE ATT&CK Framework: <https://attack.mitre.org>
- OWASP Top 10: <https://owasp.org/www-project-top-ten>

10.1.2 Key Papers (also cited in text)

- Lewis et al. (2020) - Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks: <https://arxiv.org/abs/2005.11401>
- Asai et al. (2023) - Self-RAG: Learning to Retrieve, Generate and Critique through Self-Reflection: <https://arxiv.org/abs/2310.11511>
- Shinn et al. (2023) - Reflexion: Language Agents with Verbal Reinforcement Learning: <https://arxiv.org/abs/2303.11366>
- Goodfellow et al. (2014) - Generative Adversarial Networks: <https://arxiv.org/abs/1406.2661>
- Biggio & Roli (2018) - Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning: <https://arxiv.org/abs/1712.03141>

10.1.3 Video Resources - AI & Machine Learning

- 3Blue1Brown - Neural Networks & Deep Learning: <https://www.youtube.com/@3blue1brown>
- Andrej Karpathy - Intro to Large Language Models: https://www.youtube.com/watch?v=zjkBMFhNj_g
- <https://www.youtube.com/watch?v=eur8dU09mvE>
- https://www.youtube.com/watch?v=Fhy_VFM1E9s

10.1.4 Video Resources - Fraud Detection & AI Security

- IBM Technology: <https://www.youtube.com/@IBMTechology>
- IBM Tech - Fraud Detection with AI: https://www.youtube.com/watch?v=Mo7JMC_oDlI
- ML System Design - Fraud Detection: <https://www.youtube.com/watch?v=90eszaYPskk>

10.1.5 Video Resources - AI-Powered Threats

- Generative AI Fuels Rise in Identity Fraud: <https://www.youtube.com/watch?v=2EuAfu-aPp8>
- TEDx - Dark Side of AI (Mark T. Hofmann): <https://www.youtube.com/watch?v=YWGZ12ohMJU>
- Analyst Chat 246 - Dark Side of Generative AI: https://www.youtube.com/watch?v=GycIG3w_Xds

10.1.6 Technical Documentation

- Ollama: <https://ollama.com/>
- ChromaDB: <https://docs.trychroma.com/>
- Meta Llama 3.2: <https://ai.meta.com/blog/llama-3-2/>
- Microsoft Phi4-mini: <https://huggingface.co/microsoft/Phi-4-mini-instruct>
- Mistral AI: <https://mistral.ai/>
- AWS EC2: <https://aws.amazon.com/ec2/>
- AWS S3: <https://aws.amazon.com/s3/>
- AWS CDK: <https://docs.aws.amazon.com/cdk/>
- Flask: <https://flask.palletsprojects.com/>
- SentenceTransformers: <https://www.sbert.net/>
- DVWA: <https://github.com/digininja/DVWA>
- AWS - Visualising Vector Embeddings: <https://aws.amazon.com/blogs/database/visualize-vector-embeddings-stored-in-amazon-aurora-postgresql-and-explore-semantic-similarities/>
- FastMCP: <https://fastmcp.me/>
- Model Context Protocol - Build a Server: <https://modelcontextprotocol.io/docs/develop/build-server>
- GoFastMCP - Getting Started: <https://gofastmcp.com/getting-started/welcome>